

ResumeFlow — Backend Implementation Plan

Production-oriented backend for upload → parse → edit → generate → download, with strict template preservation.

Core rule: the user's original resume design is the source of truth. Content may change; presentation must not be redesigned unless the user explicitly changes it.

1. Scope

In scope: resume upload, immutable original storage, content extraction, template/layout extraction, structured editing, versioning, generation using the original template, preview/download, PostgreSQL persistence, Redis caching.

Out of scope: ATS scoring, JD matching, AI optimization/rewrite, template marketplace, job tracking, mobile app, and authentication unless added later.

2. Stack

Layer	Choice
Language	Java 25
Framework	Spring Boot 4.x compatible with Java 25
Build	Maven
Database	PostgreSQL — database: resumeflow
Persistence	Spring Data JPA + Hibernate
Migrations	Flyway
Cache	Redis + Spring Data Redis
API	REST + multipart upload + streaming download
Documents	Apache POI / OOXML for DOCX; Apache PDFBox for PDF
Testing	JUnit 5, Mockito, Spring Boot Test, Testcontainers
Observability	Spring Boot Actuator + structured logging

3. Architecture

Use domain-oriented layers. Controllers handle HTTP only; services own business logic; repositories own persistence; parser/generator/storage are interfaces with replaceable implementations.

```
com.resumeflow ■■■ config ■■■ common ■ ■■■ exception ■ ■■■ response ■ ■■■ validation ■■■ resume ■■■
controller ■■■ dto ■■■ entity ■■■ repository ■■■ service ■■■ parser ■■■ generator ■■■ storage ■■■ mapper
```

4. Clearly Separated Services

Service	Responsibility
ResumeService	Orchestrates resume lifecycle.
ResumeUploadService	Validates upload, stores original artifact, starts parsing.
ResumeParserService	Extracts editable content and template/layout metadata.
ResumeTemplateService	Owns template preservation and layout metadata.
ResumeContentService	Updates structured content only; never changes template.
ResumeGenerationService	Combines preserved template + updated content into output.
ResumePreviewService	Produces preview/generated artifact representation.
ResumeVersionService	Creates immutable versions and tracks artifacts.

Service	Responsibility
FileStorageService	Abstracts local/S3-compatible file storage.
ResumeCacheService	Redis cache-aside reads and invalidation.
ResumeMapper	Maps entities to DTOs; no business logic.

5. Template Preservation Design

Store two representations: (1) the original immutable document and (2) a normalized editable model containing resume content plus template/layout metadata. Generation must use the original template metadata as the presentation source of truth.

DOCX: use Apache POI/OOXML to inspect and preserve paragraphs, runs, styles, tables, sections, page size, margins, headers, footers, numbering, images and relationships. Modify only mapped content nodes where possible.

PDF: use PDFBox for text blocks, coordinates, fonts, colors, images and page geometry. Arbitrary PDF round-tripping is harder; support must be explicit and integration-tested rather than claiming universal pixel-perfect editing.

Template metadata: document type, page geometry, sections, element positions, styles, source node identifiers, content mappings and original-file SHA-256 checksum.

6. PostgreSQL Model

Table	Purpose
resumes	Top-level resume record, filename, MIME type, checksum, status, timestamps.
resume_versions	Immutable version history and source/generated artifact references.
resume_contents	Structured editable content snapshot, JSONB.
resume_templates	Normalized template/layout metadata, JSONB.
resume_files	Original/generated/preview artifact metadata and storage key.
resume_sections	Optional normalized section index for querying/editing.

Use UUID primary keys, optimistic locking for concurrent edits, created_at/updated_at, database constraints, and Flyway migrations. JSONB is appropriate because arbitrary resumes contain variable sections and formatting structures.

7. REST API

Method	Endpoint	Purpose
POST	/api/v1/resumes	Upload and start parsing.
GET	/api/v1/resumes/{id}	Get editor state.
GET	/api/v1/resumes/{id}/preview	Get current preview.
PUT	/api/v1/resumes/{id}/content	Update structured content.
POST	/api/v1/resumes/{id}/generate	Generate with preserved template.
GET	/api/v1/resumes/{id}/download	Download latest generated file.
GET	/api/v1/resumes/{id}/versions	List versions.
GET	/api/v1/resumes/{id}/versions/{version}	Get a version.
DELETE	/api/v1/resumes/{id}	Delete resume and artifacts.

Use DTOs, validation, consistent HTTP status codes, request/correlation IDs and RFC 7807-style Problem Details. Never expose JPA entities directly.

8. End-to-End Business Flow

1. Controller validates multipart request and delegates to ResumeUploadService.
2. FileStorageService stores the immutable original and calculates SHA-256.
3. ResumeParserService detects DOCX/PDF and extracts content + template metadata.
4. ResumeTemplateService validates preservation metadata.
5. ResumeVersionService persists the initial version.
6. Frontend receives resume ID and editor model; large parsing operations should be asynchronous.
7. PUT content changes only editable content. Template metadata remains untouched.
8. Generate loads original/template + latest content and creates a new artifact.
9. New artifact becomes an immutable version; Redis keys are invalidated.
10. Download streams the generated artifact with correct Content-Type and Content-Disposition.

9. Redis Strategy

Use Redis cache-aside. PostgreSQL remains the source of truth. Do not store the only copy of resume data in Redis.

Key	Data	TTL / Invalidation
resume:{id}:editor	Editor DTO/content	5–15 min; evict on update
resume:{id}:template	Template metadata	30–60 min
resume:{id}:preview	Preview metadata/reference	5–15 min; evict after generation
resume:{id}:latest-version	Latest version metadata	5–15 min

Avoid caching large binary files directly. Cache metadata or artifact references unless measurements justify otherwise.

10. Production Engineering Rules

- Validate extension, MIME type, file signature and size; never trust client-provided filenames.
- Store files outside the web root with server-generated storage keys; prevent path traversal.
- Use parser timeouts and upload limits.
- Use transactions for metadata/version updates and design recovery for file-storage partial failures.
- Use constructor injection, immutable DTOs/records where practical, and no business logic in controllers.

- Use Flyway; do not use Hibernate auto-DDL in production.
- Never log resume content or sensitive document data.
- Add idempotency for retry-prone generation/upload operations.
- Use optimistic locking/version checks for concurrent edits.
- Test real DOCX/PDF samples, including multi-page and multi-column documents.

11. Environment

```
DB_HOST=localhost DB_PORT=5432 DB_NAME=resumeFlow DB_USERNAME=... DB_PASSWORD=... REDIS_HOST=localhost
REDIS_PORT=6379 RESUMEFLOW_STORAGE_PATH=./storage MAX_RESUME_SIZE_MB=10
```

12. Implementation Phases

Phase 1

Spring Boot 4 + Java 25 + Maven + configuration + Flyway + health checks.

Phase 2

Entities, repositories, PostgreSQL migrations, JSONB snapshots, versioning, optimistic locking.

Phase 3

FileStorageService, local storage provider, checksums and artifact lifecycle.

Phase 4

DOCX/PDF parser interfaces and normalized content/template models.

Phase 5

Upload, edit, version, template preservation and generation services.

Phase 6

Redis cache-aside, TTLs and invalidation.

Phase 7

REST controllers, DTOs, validation, error handling and downloads.

Phase 8

Generation quality tests with real sample resumes.

Phase 9

Unit/integration/API tests with PostgreSQL + Redis Testcontainers.

Phase 10

Replace frontend mockResumeService with Spring Boot API.

13. Definition of Done

- Original resume is stored immutably.
- Content and template/layout are extracted into separate concepts.
- Content edits cannot accidentally modify template metadata.
- Generated resume uses the preserved original template.
- Original and generated artifacts are versioned.
- Redis accelerates repeated reads while PostgreSQL remains authoritative.
- Invalid/malicious uploads are rejected safely.
- APIs have consistent validation and error responses.
- Automated tests cover core services and real documents.

■ Frontend can consume the backend without changing the existing product flow.

Critical technical caveat: exact visual preservation depends on document format. DOCX has a stronger structured round-trip path. Arbitrary PDF editing is fundamentally harder, so PDF support must be implemented and validated against representative documents before claiming pixel-perfect preservation.